

Semantic Streams Augmentation

2026 Sebastián Samaruga. <https://sebxama.blogspot.com>

This work is licensed under a [Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License](#).

Summary:

This document describes a semantic RDF based data integration and intelligence framework by means of Aggregation, Alignment and Activation of ontologies.

Base Model

```
class Resource
+ ID : URI
+ primeID : BigInteger
+ occurrences : Occurrence[]
+ apply(res: Resource[]) : Resource[]
```

```
class Occurrence extends Resource
+ player : Resource
+ context : Resource
+ role : Resource (Kind?)
```

```
class Statement extends Resource
+ context : Context
+ subject : Subject
+ predicate : Predicate
+ object : Object
```

```
class Context extends Occurrence
+ role : ContextKind
```

```
class Subject extends Occurrence
+ role : SubjectKind
```

```
class Predicate extends Occurrence
+ role : PredicateKind
```

```
class Object extends Occurrence
```

```
+ role : ObjectKind
```

```
interface Kind<PlayerType extends Occurrence,  
    AttributeType extends Occurrence,  
    ValueType extends Occurrence>
```

```
class ContextKind extends Context implements Kind<Context, Subject, Object>
```

```
class SubjectKind extends Subject implements Kind<Subject, Predicate, Object>
```

```
class PredicateKind extends Predicate implements Kind<Predicate, Subject,  
Object>
```

```
class ObjectKind extends Object implements Kind<Object, Predicate, Subject>
```

Resources

Everything is a Resource identified by an URI, assigned a Prime Number ID, keeping track of its Occurrences and thus enabling the calculation of their CPPE (Prime Number Contextual Embedding) by means of the product of its Prime Number ID by the Prime Number IDs of the other Resources in their co-Occurrences.

Occurrences

Participation of a Resource in an Occurrence in a given Role (Kind) in a given Context (Statement, Kind).

Statement Occurrences

Statements are Resources whose occurrences are their CSPOs in a given Role (Kind?)

CSPO Occurrences

Context, Subject, Predicate, Object as Resource(s). SPO occurring Statement Contexts where they are in C, S, P, O Roles.

CSPOs instances as wrapped arbitrary Resource Occurrence Context

Kinds Occurrences

As Context, Subject, Predicate, Object. Occurrences: Kind Functional Definition Statement Contexts (occurring Statement Contexts)

Kinds as Resources: Kinds occurring in Statements in C, S, P, O Roles

Statements

Context, Subject, Predicate and Object (CSPO) tuples.

Context, Subject, Predicate, Object (CSPO) Statement Occurrences have a role (Kind) in their given Statement Occurrence Context.

Kinds: ContextKind, SubjectKind, PredicateKind, ObjectKind (CK, SK, PK, OK) are themselves CSPOs respectively and can participate in a Statement Context as Occurrences.
Types of Statements:

- SPO Statements
- SPO / Kinds Statements (recursive Kinds)
- Kinds only Statements (recursive Kinds)

This is further leveraged in the Augmentation Layers Domain Model definitions section.

Contexts, Subjects, Predicates, Objects

The role of a Resource Occurrence in a Statement is typed by its Statement Occurrence position. A Resource can occur in more than one role or occurrence type (CSPO) in different Statements. The type of the role (Kind) of the Resource Occurrence is determined by the type of the position it plays in a given Statement.

Context, Subject, Predicate, Object Kinds

Aggregation of PlayerType(s) Resource Occurrences (SPOs) by common AttributeType(s) and then by common ValueTypes(s). Parameterized interface for each Kind type (CK, SK, PK, OK).

Kinds can be aggregated from raw CSPO Occurrences from any combination of C, S, P, O / CK, SK, PK, OK Statement Occurrence types. This allows for recursive Kinds definitions (folding). This is further leveraged in the Augmentation Layers Domain Model definitions section.

Kinds are a kind of “type inference”: if a Subject shares common attributes with another Subject, it is of the same Kind “type”. If their common attributes share common values, it is of the same Kind “state”.

Root Kinds are the Kinds corresponding to the CSPO Occurrences Statements positions. They have no attributes nor values, they just tag an Occurrence as being of an CSPO Occurrence position in a Statement.

Kinds hierarchies: basic inheritance, if a Subject attribute set is the superset of another Subject attribute set, it can be regarded as an “extension” of the first Subject. Leverage this with other inference mechanisms (FCA, for example).

Kinds ordering: a Subject “extended” by another Subject attributes set can be regarded as being “before” the first Subject. This assumes the existence of a super type is necessary for the existence of a sub type.

Kinds are represented and serialized by Statements whose Statement Contexts are the Kind instance itself (Kind reified Context Resource Occurrence), and their Subjects, Predicates and Objects plays the PlayerType, AttributeType and ValueType of the Kind definition. Thus, the definition of a Kind is the set of Statements of its corresponding Kind as Resource Occurrence of their Statement Contexts.

Kinds as Extensional Functional Definitions

Kind Statements can be regarded as the definition of a Function by extension. Their Statement Contexts Occurrence Resources are the Function name or signature. Their PlayerType(s) instances group their mappings by “context” and their AttributeType(s) and ValueType(s) instances are the Function domain / range corresponding value pairs. Subject

- CK: Context (Contexts), Domain (Subject), Range (Predicates):
aGivenContextKind(C, S): P
- SK: Context (Subjects), Domain (Predicates), Range (Objects):
aGivenSubjectKind(S, P): O
- PK: Context (Predicates), Domain (Subjects), Range (Objects):
aGivenPredicateKind(P, S): O
- OK: Context (Objects), Domain (Predicate), Range (Subject): aGivenObjectKind(O, P): S

Kinds based functional merge:

If $aKind(S1) = A$, and $aKind(S2) = A$, S1 and S2 are equivalent in a given context.

If $aKind(S1) = A$, $aKind(next(S1)) = next(A)$.

Functional Composition (align, sort, link completion, merge).

Kinds Functional Comprehensions:

Kinds: extensional functions definitions. Functions Comprehensions: infer Kinds Functions from CPPE embeddings via Helpers / ML. Resource Merge / Apply: Kinds inferred Functions inference.

Base Model Representations

Base Model class hierarchy is represented by different meta model approaches (kept in sync with each other) for leveraging the most appropriate inference mechanisms.

Models persistence Monadic wrapper bound Base Model Entities (Resource, Occurrence, Statement, Context, Subject, Predicate, Object, Kind, ContextKind, SubjectKind, PredicateKind, ObjectKind) Functional APIs Definitions for stream processing.

RDF Model Representation

Raw RDF Statements Triple (Quad) Store. SPARQL Endpoint.

Sets Based Model Representation

Contexts Set (Subjects Set, Predicates Set and Objects Set union).

Subjects Set.

Predicates Set.

Objects Set.

Context Kinds Set (Subject Kinds Set, Predicate Kinds Set, Object Kinds Set union).

Subject Kinds Set (Predicates Set intersection with Objects Set).

Predicate Kinds Set (Subjects Set intersection with Objects Set).

Object Kinds Set (Subjects Set intersection with Predicates Set).

ISO TMRM Model Representation

Underlying common metamodel (TODO).

FCA Context Lattices Model Representation

Context, Subject, Predicate, Object as Context Formal Contexts (TODO).

XML Model Representation

TreeGraph for nested Statements structures Stream Processing. Pipeline messages format. See "Messaging Infrastructure".

Execution Model: Augmentation Agents Pipeline

A sequential message driven execution model, the Augmentation Pipeline, performs, by means of message exchange streams processing (unfolding, merging and folding), the Semantic Streams Augmentation of raw RDF CSPO URIs Quads input Statements (from configurable Datasources*) into executable models of knowledge gathered by means of (1)

Aggregation, (2) Alignment and (3) Activation of Statements (messages) encoding corresponding layers domain models Statement types structures.

Augmentation Agents communicate with each other by means of a series of messaging endpoints where each layer consumes its input (source) Statements and publishes their output (production) Statements. Layers input and output Statements are of two types: an Augmentation Agent layer can consume both of their source Statements types and their production Statements types.

In the first case, processing a source Statement type input, it “merges” this input source type Statement with its source level type already known processed Statements aggregating, aligning and activating possible new inferred Statements. It then “folds” (aggregates) the resulting merged Statements into its layer corresponding Kind types Statements, then builds and produces its output production (Kinds) Statement types for production publishing.

In the case of a layer’s production Statement type as the consumed input, agents merges this input with its already known production statement types (matching / merging input Statement Kinds with already known Kinds), it then “unfolds” input Statement components (Kinds) into layer’s Kinds aggregated originating source type Statements obtaining this way the source Statement type Statements which originated the input (Kinds) production Statement message (source Statements). It then performs, with each “unfolded” Statement, the same way as if it were like those source Statements where consumed source Statements (merging, folding and publishing). Intermediate source type merged new inferred Statements are also merged, folded and published.

This Agent pipeline execution model could be depicted as:

source statements input / merge -> folding -> production statements merge and publishing

production statements input / merge -> unfold -> source statements merge and publishing

The merge steps in the pipeline is where new knowledge (Statements) are Aggregated, Aligned and Activated, where pipeline layers inference is performed materializing inferred knowledge into new Statements of each type. That is why it makes sense to merge new inputs with previous knowledge and inferred knowledge in a pipeline in / out fashion.

This fan in / fan out approach is intended to be implemented in a reactive functional streams pipeline where each agent consumes and produces knowledge asynchronously and in parallel. An Agent production type Statement is another Agent source type Statement (maybe including the Agent itself) and an Agent source type Statement is the type of another Agent production Statement type. An Agent “accepts” an Statement (source or production) if it is not known already or if it was further processed (updated) its context since the last consumption (or “acceptance” performance is no-op).

Homoiconic (code as data) approach

Messaging pipeline oriented resources consumption and publishing. Merge, Fold, Unfold phases functional approach.

Merging input data against previously known data, applying previously known data as a Template / Alignment / Transformation for merge over input data is what is meant with “code as data”.

Resources (Occurrences, Statements, Contexts, Subjects, Predicates, Objects and their corresponding Kinds) are “functional” entities that can be “applied” to another Resources. The signature of such functional composition is as follows:

```
Resource::apply(res: Resource) : Resource
Resource::apply(res: Occurrence) : Resource
Resource::apply(res: Statement) : Resource
Resource::apply(res: Context) : Resource
Resource::apply(res: Subject) : Resource
Resource::apply(res: Predicate) : Resource
Resource::apply(res: Object) : Resource
Resource::apply(res: ContextKind) : Resource
Resource::apply(res: SubjectKind) : Resource
Resource::apply(res: PredicateKind) : Resource
Resource::apply(res: ObjectKind) : Resource
```

Each Resource hierarchy class has their own overridden implementation of those ‘apply’ methods. The semantics of such “applications” (argument types and return types) are to be defined for each Resource type subclass implementing or overriding the corresponding ‘apply’ methods, their internal state and leveraging the following by means of Helper Services:

Apply as functional graph traversal by invocations compositions.

Kinds Functional Application.

FCA Inference / Traversal.

CPPE Contextual Inference.

Sets Model Representation.

Example: Kind applied to a Resource returns all Statements where the Resource is a player of the Kind.

Given a monadic wrapper (Spring Flux, Mono, for example) for handling streams processing:

```
aWrappedResource.map(aResourceInstance::apply) :
Flux<AResourceInstanceApplyType>
```

Then, Resource(s) applications can be chained and combined by means of stream pipelines composition.

Kinds Functional Comprehensions:

Kinds: extensional functions definitions. Functions Comprehensions: infer Kinds Functions from CPPE embeddings via Helpers / ML. Resource Merge / Apply: Kinds inferred Functions inference.

Monadic fold / unfold:

Aggregate (Kinds map / reduce):
key (PlayerType), values (AttributeType, ValueType).

De aggregate: unfold to original Kind Statements (stream scan).

Merge: application of aggregated (folded) / de aggregated (unfolded) Resources to output / input streams (augmented with previous merge results) on each key (fold) aggregation completion and on each unfold stream scan emitted Resource.

TODO: Resource::apply Helpers Services approaches: FCA, CPPE, RDF inference, Sets, etc. ML Models / LLMs (classification : Aggregation, clustering : Alignment, regression : Activation). Unify Augmentation application Helper Services approaches APIs.

Data Structures (Folded Kinds):

```
<Map<Resource, Set<Map<Resource, Set<Resource>>>>>
```

```
<Map<Subject, Set<Predicate>>  
<Map<Set<Predicate>, Type>  
<Type, Set<Map<Predicate, Value>>>  
<Map<Set<Map<Predicate, Value>>, State>
```

Materialization of Resources: State / IO

TODO: Refactor to IO / State Monad. Execution Resource Statements Schema.

“Primitive” Resources (Kinds) shall exist which allows for executable composition of streams models by their types and their internal instance state. Their signature may be a composite traversal of application invocations. For example, for the “createLink” Resource primitive could be invoked like:

```
createLinkResource::apply(subjectRes : Resource)::apply(predicateRes :  
Resource)::apply(objectRes : Resource) : Statement Resource
```

Merge by means of Resource Application

Merge phase of pipeline agents processing first applies its input source / production Statement to each corresponding already known Statement. It then recursively applies resulting / inferred application result Resource(s) to the corresponding already known Statement structure composition.

Folding / Unfolding by means of Resource Application

Kinds layers (aggregation Statement types) folding and unfolding may be also defined in terms of Resource(s) functional application. Then the streaming pipeline can be built in terms of those operations (merge, fold and unfold) with streaming flows corresponding to each Augmentation pipeline step.

In this way, folding and unfolding would become “contextualized” operations, where each application’s context is the previously applied Resource.

Persistence and Endpoints Interactions by means of Resource Application

TODO: A schema shall exist which encodes Persistence IO actions and Endpoints Context Interactions behaviors and executions into Execution Statements Materialization / State IO performances.

TODO:

Resource hierarchy application semantics. Full layers types hierarchy (implement all layers models kinds). Template methods. Implementation Language? (Resources, Streams Processing), XSLT?

Comparisons: Dimensional axis Order and Matching

Folding and unfolding Merge operations can leverage comparisons of Resources for sorting and matching. This is sorting in a nested contexts approach where one sorted occurrence may be the ‘parent’ of child sorted occurrences (happened and sorted in the parent occurrence scope). Resource application can be used for sorted structures traversal and further comparisons.

TODO: For leveraging CPPE, Algebraic Embeddings and FCA the comparison results may be encoded in a 3 bit string mask (octal) representation, for example: 001 is less than, 100 is greater than, 111 is equal, etc.

One goal would be to be able to encode comparison results as Merge Materialized Merge Traversal Comparison Statements Results. Example: A to B in Context C is like D to E in Context F.

The means for being able to construct such Statements for comparisons is by means of PredicateKind. Reified Relationships aggregation:

(:subject, :employer / :position / :salary, :objects...)

-> (:subject, :employment, :anEmployment)

-> (:anEmployment, :attributes..., :values...)

:employment: WorkRelationship PredicateKind (materialize)

:anEmployment: Employment Relationship Resource instance Kinds (ObjectKind, SubjectKind).

Attributes: Dimensional Axes. Values: PredicateKind ValueType Aggregation. Infer and materialize :anEmployment reified relationship object / subject for :employment predicate of WorkRelationship PredicateKind.

PredicateKind of SubjectKind Kind Attribute (Employee, Student) / ObjectKind Kind Value (Employer, School).

TODO: Infer

Employee is to Employer (in WorkRelationship) as Student is to School (in StudentshipRelationship).

Materialize: (Resource, ComparisonEncoding(axis, binary mask), Resource).

Merge (materialize) Resources according to their axes binary mask comparison results.

Leverage CPPE, Algebraic Embeddings, FCA.

Augmentation Layers Kinds Relationships Reification

Augmentation Layers, particularly the Information or Alignment layer already performs such a PredicateKind relationship, aggregating those Kinds from Subject and Object Kinds. Knowledge Activation Layers performs the aggregation over the previously aggregated layer results. Such arrangements should allow for: Information Alignment axes ordering and matching and Knowledge Activation axes ordering and matching.

The examples shown so far may be appropriate for Information Alignment Layer instance data comparisons and inference. Knowledge Activation layer should materialize ordering and matching of reified contexts and interactions of behavior instance data.

Messaging Infrastructure

Messaging pipeline:

aggregationSourceTopic <-> Aggregation <-> aggregationProductionTopic

aggregationProductionTopic <-> Alignment <-> alignmentProductionTopic

alignmentProductionTopic <-> Activation <-> activationProductionTopic

Messaging Format: TreeGraph

All Base Model entities Statements of each layer type must encode all of their corresponding composing entities in a single message. A Statement containing a specific Kind must encode all its Kind's composing aggregation Statements, for example.

Source, Production Messages includes all Statements hierarchy which compose / lead to produced Statements (TreeGraph). Serialization. References to Resources (Registry Helper Service) to reduce message payload size..

Datasources

Raw RDF CSPO URIs Quad Statement Messages Endpoint. Configured to produce configured datasource data and to consume updated Statements synchronizing state.

Stream-Oriented Merge, Fold and Unfold of Layered Statements

As Resource functional composition (Homoiconic approach).

Layers inputs / outputs. Layers functional definitions. Rules from source 'terminals' to production 'non-terminals': produce all possible production source Statements given contexts / constraints. Inferences.

TODO

Fold and Unfold as the Stream Context Resolution Operation

TODO

Merge as the Central Stream Contexts Operation

Inferred new Statements by Augmentation pipeline phase.

TODO

Possible Merge Approaches

Identity Resolution driven. TODO: Find the way of unambiguously identifying concepts (Resources, Occurrences, Statements, SPOs, Kinds) equivalent (in a given role occurrence in a given context).

Merge: Reify as Resources Occurrences, Statements, SPOs, Kinds. Resolve ID of Resources, Contexts (ID, Type), Occurrences (IDs, Contexts, Roles).

Aggregated Processing / Production Statements acts as Templates / Upper ontology alignment schemas.

Layers context merged entities propagate upstream (unfolding) and downstream (folding).

Identity Encoding:

CPPE Embeddings

Index API.

FCA / Set Assertion Statements:

(Context, Concept, Role, Occurrence);

Concept, Context, Role, Occurrence recursively instance of Set Statements.

Naming Scheme:

Name assignment for inferred entities.

Registry Mapping of Merge encodings.

RDF Merge

This approach relies on graph theory. It essentially performs a graph union of multiple RDF datasets. A critical function of an RDF merge is the standardization and renaming of "blank nodes" (anonymous resources) across different graphs to avoid collisions, resulting in a single, unified knowledge graph of subject-predicate-object triples.

Merge Statements as RDF graph patterns, combining compatible subjects, predicates, and objects while preserving graph identity and provenance. This provides a straightforward representation for stream-level graph merging and interoperability with RDF-based tooling.

TMRM Merge

TMRM merging is driven strictly by subject identity. If two topics (entities) across different data streams share the same subject locator, subject identifier, or item identifier, they are deterministically merged into a single topic. The resulting merged topic accumulates all the names, occurrences, and associations of the original topics.

Perform merging at the TMRM metamodel level, using Resources, Occurrences, Statements, Subjects, Predicates, Objects, and Kinds as the semantic structures being matched and consolidated. This provides the underlying subject-centric merge and identity model for the pipeline.

FCA Merge

This approach merges data at the conceptual level by combining "formal contexts" (mappings of objects to their attributes). When two concept lattices are merged, the FCA algorithm recalculates the matrix to build a new, unified lattice. This is highly effective for discovering new structural hierarchies and implicit relationships that weren't visible in the isolated data streams.

Treat matching Statements as formal contexts and merge compatible contexts to derive shared Properties and higher-level relationships. This is particularly suited to the Aggregation phase, where input Statements are consolidated into Property Statements.

CPPE Based FCA / Model Primitives Merge

Operating at the most atomic level of a data architecture, this approach aligns the fundamental structural primitives (e.g., core entities, base properties, and semantic rules) of the models. By merging the structural primitives *first*, the pipeline creates a standardized baseline context, which is then fed into an FCA merge to build highly accurate, normalized concept lattices.

Combine FCA context merging with CPPE/model primitives to support higher-level semantic matching and inference. This approach can be used in the Alignment phase to derive ontology matches, links, dimensions, ordering, and related Rule Statements.

Sets Model Based Merge

Rooted in classic set theory, this methodology uses strict mathematical operations (unions, intersections, and relative differences) to combine data. It treats data streams as collections of elements, making it an incredibly fast, deterministic, and highly scalable approach for deduplication and exact-match aggregations.

Represent Statement components and their relationships as sets and apply set operations such as union, intersection, difference, and compatibility matching. This provides a simple compositional mechanism that can be used as a lower-level merge primitive within the other approaches.

Integration into Streams Processing

TODO

Streaming Agents Augmentation Pipeline

Each pipeline Agent has three types of Statements: its Source Statements (consumes / produces), its internal Kind aggregation Statements (used in the merge phase of its processing) and its Production Statements (produces / consumes).

Any of the three layers can be considered as (sets of) Resources of their types of source and production Statements. Layer state. Streams application processing (layer endpoints, topics).

Aggregation (Data) Agent:

Source Statement Type: Raw RDF CSPO URIs Quad Statements.

Production Statement Type: (CK, SK, PK, OK) Property Quad Statements.

Alignment (Information) Agent:

Source Statement Type: (CK, SK, PK, OK) Property Quad Statements.

Production Statement Type: (CK, SK, PK, OK) Rule Quad Statements.

Activation (Knowledge) Agent:

Source Statement Type: Actor Statements (Alignment Information Layer outputs)

Production Statement Type: (CK, SK, PK, OK) Context Quad Statements.

TODO: Cleanup Agents Domain Model Examples.

Aggregation Service Agent

Context / Role / Type inference in the Aggregation Layer.

Aggregation Agent Domain Model

Data Layer. DOM (Dynamic Object Model)

Data / Instance layer input Statements:

Source Statement Type. Raw CSPO RDF URIs Quad Statements.

(C, S, P, O)

Type aggregation Statements:

Kinds aggregation Statement Types.

(ContextCK, S, P, O)

Context Kind, PlayerType: Context, AttributeType: Subject, ValueType: Object.

Example: (EmploymentCK, :Peter, :worksFor, :anEnterprise)

(TypeSK, S, P, O)

SubjectKind, PlayerType: Subject, AttributeType: Predicate, ValueType: Object.

Example: (EmployeeTypeSK, :Peter, :employer, :anEnterprise)

(PropertyPK, S, P, O)

Predicate Kind, PlayerType: Predicate, AttributeType: Subject, ValueType: Object.

Example: (EmploymentRelationshipPropertyPK, :Peter, :worksFor, :anEnterprise)

(ValueOK, S, P, O)

Object Kind, PlayerType: Object, AttributeType: Predicate, ValueType: Subject.

Example: (EmployerValueOK, :Peter, :employedBy, :anEnterprise)

Property output Statements:

Production Statement Type.

(ContextCK, TypeSK, PropertyPK, ValueOK)

Example:

(EmploymentContextKind, EmployeeSubjectKind, EmploymentRelationshipPredicateKind, EmployerObjectKind)

Aggregation Agent Merge / Fold / Unfold

See Homoiconic approach. TODO

Alignment Service Agent

Ontology Matching / Links / Attributes / Dimensional / Order inference in the Alignment Layer.

Alignment Agent Domain Model

Information Layer. N-ary Dimensional Relationships Modelling.

Axis layer input Statements:

Source Statement Type. Aggregation Production Statements.

(C, S, P, O)

(ContextCK, TypeSK, PropertyPK, ValueOK) from Aggregation Property Statements.

Example:

(EmploymentContextKind, EmployeeSubjectKind, EmploymentRelationshipPredicateKind, EmployerObjectKind)

State aggregation Statements:

Kinds aggregation Statement Types.

(RelationshipCK, TypeSK, PropertyPK, ValueOK)

Context Kind, PlayerType: ContextCK, AttributeType: TypeSK, ValueType: ValueOK.

Example: (EmploymentRelationshipCK, EmployeeSubjectKind, EmploymentRelationshipPredicateKind, EmployerObjectKind)

(SourceEndSK, TypeSK, PropertyPK, ValueOK)

Subject Kind, PlayerType: TypeSK, AttributeType: PropertyPK, ValueType: ValueOK.

Example: (EmployeeRoleSK, EmployeeSubjectKind, EmploymentRoleRelationshipPredicateKind, EmployeeRoleObjectKind)

(MeasurementContextPK, TypeSK, PropertyPK, ValueOK)

Predicate Kind, PlayerType: PropertySK, AttributeType: TypePK, ValueType: ValueOK.

Example: (EmployeePositionAtDatePK, EmployeeSubjectKind, EmploymentPositionRelationshipPredicateKind, EmployeePositionObjectKind)

(DestinationEndOK, TypeSK, PropertyPK, ValueOK)

Object Kind, PlayerType: ValueOK, AttributeType: PropertyPK, ValueType: TypeSK.

Example: (CurrentEmployeeRoleOK, EmployeeSubjectKind, EmploymentRoleRelationshipPredicateKind, EmployeeRoleObjectKind)

Rule output Statements:

Production Statement Type.

(RelationshipCK, SourceEndSK, MeasurementContextPK, DestinationEndOK)

Example:

(EmploymentRelationshipCK, EmployeeRoleSK, EmployeePositionAtDatePK, CurrentEmployeeRoleOK)

Alignment Agent Merge / Fold / Unfold

See Homoiconic approach. TODO

Activation Service Agent

Contexts (Use Cases) and behaviors (Interactions) in the Activation Layer.

Activation Agent Domain Model

Knowledge Layer. DCI Contexts, Roles, Actors, Interactions.

Actor layer input Statements:

Source Statement Type. Alignment Rule Statements.

(C, S, P, O):

(RelationshipCK, SourceEndSK, MeasurementContextPK, DestinationEndOK) from Alignment Rule Statements.

Example:

(EmploymentRelationshipCK, EmployeeRoleSK, EmployeePositionAtDatePK, CurrentEmployeeRoleOK)

Role Interaction aggregation Statements:

Kinds aggregation Statement Types.

(DCIContextCK, SourceEndSK, MeasurementContextPK, DestinationEndOK)

Context Kind, PlayerType: DCIContextCK, AttributeType: MeasurementSK, ValueType: MeasurementOK.

Example: (EmploymentDepartmentCK, EmploymentDepartmentMeasurementSK, EmploymentRoleDimensionPK, EmploymentRoleMeasurementOK)

(RoleSK, SourceEndSK, MeasurementContextPK, DestinationEndOK)

Subject Kind, PlayerType: RoleSK, AttributeType: DimensionPK, ValueType: MeasurementOK.

Example: (EmploymentDepartmentRoleSK, EmploymentDepartmentMeasurementSK, EmploymentRoleDimensionPK, EmploymentRoleMeasurementOK)

(ActorPK, SourceEndSK, MeasurementContextPK, DestinationEndOK)

Predicate Kind, PlayerType: DimensionPK, AttributeType: MeasurementSK, ValueType: MeasurementOK.

Example: (EmploymentDepartmentActorPK, EmploymentDepartmentMeasurementSK, EmploymentRoleDimensionPK, EmploymentRoleMeasurementOK)

(InteractionOK, SourceEndSK, MeasurementContextPK, DestinationEndOK)

Object Kind, PlayerType: MeasurementOK, AttributeType: DimensionPK, ValueType: MeasurementSK.

Example: (EmploymentDepartmentRoleOK, EmploymentRoleMeasurementSK, EmploymentRoleDimensionPK, EmploymentRoleMeasurementOK)

Context output Statements:

Production Statement Type.

(DCIContextCK, RoleSK, ActorPK, InteractionOK)

Example:

(DCIContextCK, EmploymentDepartmentRoleSK, EmploymentDepartmentRoleSK, EmploymentDepartmentRoleOK)

Activation Agent Merge / Fold / Unfold

See Homoiconic approach. TODO

Helper Services

Monadic wrapper bound Base Model Entities (Resource, Occurrence, Statement, Context, Subject, Predicate, Object, Kind, ContextKind, SubjectKind, PredicateKind, ObjectKind)
Functional APIs Definitions for stream processing.

“Leverage X” Sections Functional APIs.

Kinds as Extensional Function Definitions Functional APIs.

Resource merge / “application” (transform / template) Functional APIs.

Registry Service

Registry API Functional Definitions.

Naming Service

Naming API Functional Definitions.

Index Service

Index API Functional Definitions.

Dynamic API Endpoints

HAL like or discoverable behavior APIs over Activation activated knowledge statements.
Render interfaces for contexts interactions instances. Message driven API rendering
Contexts Interactions (DCI) behaviors (state) and role executions (verbs).

Leverage TMRM

Underlying graph metamodel.

Use the ISO Topic Maps Reference Model (TMRM) as the underlying graph metamodel for the Base Model, providing a subject-centric and implementation-independent foundation for representing **Resource**, **Occurrence**, **Statement**, **Subject**, **Predicate**, **Object**, and their associated **Kind** structures. The Base Model can be mapped onto TMRM subject proxies and properties, with Statements providing the reified graph relationships and Occurrences providing the contextual roles in which Resources participate.

TMRM can therefore provide the semantic graph substrate and generic graph operations required by the service, while the Base Model and Augmentation Pipeline define the application-specific semantics and processing layers. In particular, the metamodel should support APIs for:

- **Resource / subject management:** create, identify, retrieve, update and merge graph resources/subject proxies, including stable URI-based identity.
- **Statement management:** create, retrieve, remove and traverse reified **S-P-O** Statements and their contextual Occurrences and roles.
- **Graph traversal:** navigate subjects, predicates, objects, occurrences, contexts and their relationships, including recursive traversal of Kinds and Statements.
- **Context-preserving access:** retrieve the statements and properties associated with a subject while preserving the context in which each relationship or occurrence was asserted.
- **Property and kind access:** expose the properties of a resource and the Statements defining **SubjectKind**, **PredicateKind** and **ObjectKind**, supporting the classification structures used by the Aggregation phase.
- **Pattern/query access:** match graph structures and Statement patterns so that Augmentation Pipeline agents can discover applicable processing Statements and apply them to input Statement streams.
- **Identity and merging:** provide subject identity/merging facilities that allow information about the same underlying subject from different sources or streams to be consolidated without losing its provenance/context.
- **Schema/constraint support:** expose the metamodel structures needed by higher-level services to validate or constrain the graph, while leaving the specific application constraints to the service model.

The TMRM layer is thus primarily responsible for **representing and accessing the semantic graph**, rather than implementing the Aggregation, Alignment or Activation semantics themselves. Those services consume and produce the graph structures through the metamodel APIs, allowing the same underlying graph representation to support the DOM, Dimensional and DCI layers described by the Augmentation Pipeline.

This also provides a natural foundation for the existing FCA and CPPE components: FCA can operate over the subject/property structures exposed by the graph, while CPPE can use Resource/Statement identities and identifiers as inputs to its numerical inference mechanisms.

Leverage Dimensional Features

Statement Predicates are the Type or Instance of an Action / Verb

:name instanceOf :naming

The Power Set of every attribute contains the sets of attributes of every possible class / concept. Includes relation defines an ordering over the power set. Power Sets (CSPO / Reified Types Sets). All possible Subjects, Predicate, Objects, Kinds Grouped Types. Order Relationship (Includes). CSPO Sets Intersection: All possible Statements. Grammar (production from possible rules).

Order and hierarchies are defined by subset / superset relationships. An object having attributes who are a superset of another object attributes is “extending” or “after” this object. Dimensional (Contexts) Features

Kinds, type / state hierarchy / order inference and materialization into Statements.

Alignment. Order (axis arrangements). State transitions flow. Infer and encode types / instances state transition networks on a given context on a given event:

- PCN (Previous, Current, Next) in axis / scope Graph Traversal Node Types: Individuals, States (occurrences role types), Associations (events). Order Hierarchies. Contexts:
 - (Single, Marriage, Married);
 - (Marriage, Divorce, Marriage);
 - (Single, Married, Divorced);
 - (Junior, Promotion, Semisenior);
 - (Junior, Semisenior, Senior);
 - (Unemployed, Employment, Employee);
 - (Unemployed, Employed, Unemployed);
 - (John, successor, Peter);

Leveraging Grammars as an LLM Interaction tool

Possible contexts / functional transitions (behavior) as formal grammars rules / productions. Possible Prompts in context. Contexts Roles behaviors Flows. Inference context productions from grammars. Statements as grammars rules / productions.

Leverage RDF

TODO

Leverage Sets Model

TODO

Leverage FCA

Formal Concept Analysis for classification, types and attributes inference.

Subjects as Contexts:

Objects: Statement Predicates

Attributes: Statement Objects

Predicates as Contexts:

Objects: Statement Subjects

Attributes: Statement Objects

Objects as Contexts:

Objects: Statement Subjects

Attributes: Statement Predicates

Formal Concept Analysis (FCA) can be used as an inference model over the Base Model to derive classifications, types, attributes and shared properties from the Resource/Occurrence/Statement structures. FCA should operate on the graph structures exposed by the TMRM layer, treating the reified Statements as the source of object–attribute incidence relations rather than introducing a separate representation.

The three FCA contexts defined above provide complementary inference views:

- **Subject context:** Subjects are FCA contexts, Statement Predicates are objects, and Statement Objects are attributes. This allows the system to infer concepts representing groups of Subjects sharing predicate/object characteristics.
- **Predicate context:** Predicates are contexts, Statement Subjects are objects, and Statement Objects are attributes. This can infer common domains, ranges and behavioral/property patterns for predicates.
- **Object context:** Objects are contexts, Statement Subjects are objects, and Statement Predicates are attributes. This can infer classifications of objects based on the predicates through which they occur.

FCA-based Relational Schema Inference:

The system can infer relational schemas (rules or "upper concepts") from the structure of the data itself using FCA.

- **FCA Contexts for Relational Analysis:** We use three types of FCA contexts to analyze relationships from different perspectives:
 1. **Predicate-as-Context:** (FCA Objects: Statement Subjects, FCA Attributes: Statement Objects, FCA Context: Statement Predicate). This context reveals which types of subjects relate to which types of objects for a given predicate.
 2. **Subject-as-Context:** (FCA Objects: Statement Predicates, FCA Attributes: Statement Objects, FCA Context: Statement Subjects). This reveals all the relationships and objects associated with a given subject, defining its role.
 3. **Object-as-Context:** (FCA Objects: Statement Subjects, FCA Attributes: Statements Predicates, FCA Context: Statement Objects). This reveals all the

subjects and actions that affect a given object.

- **Algorithm: Inferring Relational Schema:**

1. **Select Context:** For a given predicate P (e.g., :worksOn), the Alignment Service constructs the Predicate-as-Context.
2. **Build Lattice:** It uses an FCA library (e.g., fcalib) to compute the concept lattice from this context.
3. **Identify Formal Concepts:** Each node in the lattice is a formal concept (A, B), where A is a set of subjects (the "extent") and B is the set of objects they all share (the "intent").
4. **Materialize Schema:** Each formal concept represents an inferred relational schema or "upper concept". The system creates a new RDF class for this concept. For a concept where the extent is {dev1, dev2} (both :Developers) and the intent is {projA, projB} (both :Projects), the system can materialize a schema:
:DeveloperWorksOnProject a rdfs:Class, :RelationalSchema ;
:hasDomain :Developer ;
:hasRange :Project .

Leverage CPPE

Algebraic inference by means of numerical prime numbers IDs embeddings.

Algebraic Semantic Embeddings (CPPE):

Prime Numbers Identifiers:

Every Resource is assigned a unique Prime Number ID (sequence) associated with its URI (via Registry Helper Service API).

Prime ID Embeddings:

Each Resource is assigned an unique incremental Prime Number Identifier.

For a given Resource occurrence in a given Statement context its Prime ID Embedding is calculated as the product of this occurrence Resource Prime ID Embedding with the Prime ID Embeddings of the other two parts of the Statement occurrences.

For example: given an object occurrences its Prime ID Embedding is the product of its Prime ID (Embedding) by the Prime ID (Embedding) of the other occurrences components.

Augmentation Layers. Stream Pipelines:

Aggregation, Alignment, Activation steps. Leverage Prime ID Embeddings for reactive functional composition.

CPPE. FCA-based Embeddings: A Deterministic Approach:

Resource Embeddings are calculated by the product of the Resource's Prime ID by the product of the Resource's Occurrences Attributes and Values Prime IDs.

We will replace LLM-based embeddings with deterministic, structural embeddings derived from FCA contexts and prime number products. This provides explainable similarity based on shared roles and relationships.

- **Contextual Prime Product Embedding (CPPE):** For any Occurrence (i.e., a resource in a specific statement), we can calculate an embedding based on its relational context.
 1. **Define FCA Contexts:** For a given relation (predicate), we can form an FCA context. Example: For the predicate `:worksFor`:
 - **Objects (G):** The set of all subjects of `:worksFor` statements (e.g., `{id:Alice, id:Bob}`).
 - **Attributes (M):** The set of all objects of `:worksFor` statements (e.g., `{id:Google, id:StartupX}`).
 2. **Calculate Prime Product:** The CPPE for `id:Google` within the `:worksFor` context is the product of the primeIDs of all employees who work there. $CPPE(Google, worksFor) = primeID(Alice) * primeID(Bob) * \dots$
- **Similarity Calculation & Inference:**
 - **Similarity:** The similarity between two entities in the same context is the Greatest Common Divisor (GCD) of their CPPEs. $GCD(CPPE(Google), CPPE(StartupX))$ reveals the primeID product of their shared employees, giving a measure of personnel overlap.
 - **Relational Inference:** We can infer complex relationships. Consider the goal of finding an "uncle".
 1. Calculate the CPPE for "Person A" in the `:brotherOf` context (the product of their siblings' primes).
 2. Calculate the CPPE for "Person B" in the `:fatherOf` context (the product of their children's primes).
 3. If $GCD(CPPE_brotherOf(A), CPPE_fatherOf(B)) > 1$, it means A is the brother of B's father. The system can then materialize a new triple: (A, `:uncleOf`, `ChildOfB`). This inference is stored and queryable.

Relational Context Vectors

The core of this approach is the Relational Context Vector (RCV). For any given statement (a reified triple), we compute a vector of three BigInteger values, (S, P, O). Each component is a CPPE calculated from one of the three FCA context perspectives, providing a holistic numerical signature of the statement's role in the graph.

- **RCV Definition:** $RCV(statement) = (S, P, O)$
 - **S (Subject Context Embedding):** The CPPE of the statement's subject from the Subject-as-Context perspective. This number encodes everything the subject does. $S = calculateCPPE(statement.subject, SubjectAsContext)$
 - **P (Predicate Context Embedding):** The CPPE of the statement's predicate from

the Predicate-as-Context perspective. This number encodes every subject-object pair the predicate connects. $P = \text{calculateCPPE}(\text{statement.predicate}, \text{PredicateAsContext})$

- **O (Object Context Embedding):** The CPPE of the statement's object from the Object-as-Context perspective. This number encodes everything that happens to the object. $O = \text{calculateCPPE}(\text{statement.object}, \text{ObjectAsContext})$
- **Implementation:** A Java record `RCV(BigInteger s, BigInteger p, BigInteger o)`. The Index Service is responsible for calculating and caching the RCV for every reified statement in the graph.

Schema Archetypes:

This dual representation is key to performing inference.

- **Instance RCV:** The RCV calculated for a specific, concrete statement (e.g., `stmt_123: (dev:Alice, :worksOn, proj:Orion)`) is its unique numerical signature. It represents a single data point.
- **Schema RCV (Archetype):** The RCV for a relational schema (e.g., the `:DeveloperWorksOnProject` schema) is an "archetype" vector. It is calculated by finding the Least Common Multiple (LCM) of the corresponding components of all instance RCVs that belong to that schema.
 - **Algorithm: calculateSchemaRCV(schemaURI)**
 1. Find all instance statements `s_i` where `s_i rdf:type schemaURI`.
 2. For each instance `s_i`, retrieve its cached `RCV_i = (S_i, P_i, O_i)`.
 3. Calculate the schema RCV components:
 - $S_{\text{schema}} = \text{LCM}(S_1, S_2, \dots, S_n)$
 - $P_{\text{schema}} = \text{LCM}(P_1, P_2, \dots, P_n)$
 - $O_{\text{schema}} = \text{LCM}(O_1, O_2, \dots, O_n)$
 4. The result $(S_{\text{schema}}, P_{\text{schema}}, O_{\text{schema}})$ is the numerical archetype for the schema. The LCM ensures that the schema's numerical signature is "divisible" by all of its instances.

Subsumption:

Subsumption / Instance Checking (rdf:type):

- **Concept:** An instance belongs to a schema if the instance's RCV "divides into" the schema's RCV.
- **Algorithm: isInstanceOf(instanceRCV, schemaRCV)**
 1. Perform a component-wise modulo operation.
 2. `boolean isS = schemaRCV.s.mod(instanceRCV.s).equals(BigInteger.ZERO);`
 3. `boolean isP = schemaRCV.p.mod(instanceRCV.p).equals(BigInteger.ZERO);`
 4. `boolean isO = schemaRCV.o.mod(instanceRCV.o).equals(BigInteger.ZERO);`
 5. Return `isS && isP && isO`.
- **Use Case:** This is a high-speed, purely numerical method for checking type constraints,

which can be performed in memory without a complex graph query.

Property Chains

Define composed relations (e.g., knowsLanguage) and leverage reasoners for closure. This section details the specific numerical algorithm for the $(:Developer)-[:worksOn]->(:Project)$ and $(:Project)-[:usesLanguage]->(:Language) \implies$ $(:Developer)-[:knowsLanguage]->(:Language)$ inference.

- **Step 1: Define the Composition Operator**
compose(RCV1, RCV2)
 - **Inferred Subject (S_inferred):** $S_inferred = RCV1.s * RCV2.o$
 - **Inferred Object (O_inferred):** $O_inferred = RCV1.s * RCV2.o$
 - **Inferred Predicate (P_inferred):** $P_inferred = RCV1.p * RCV2.p$
- **Step 2: Calculate Schema Archetypes**
 - The Index Service calculates archetypal RCVs for source schemas:
 - $RCV_worksOn_schema = (S_wo, P_wo, O_wo)$
 - $RCV_usesLang_schema = (S_ul, P_ul, O_ul)$
 - It then calculates the archetypal RCV for the inferred schema (knowsLanguage):
 - $S_kl = S_wo * O_ul$
 - $P_kl = P_wo * P_ul$
 - $O_kl = S_wo * O_ul$
 - This resulting $RCV_knowsLang_schema = (S_kl, P_kl, O_kl)$ is stored.
- **Step 3: The Inference Algorithm at Query Time**
A user asks: "Does dev:Alice know lang:Java?"
 1. **Retrieve Instance RCVs:** Retrieve RCV1 for (dev:Alice, :worksOn, proj:Orion) and RCV2 for (proj:Orion, :usesLanguage, lang:Java).
 2. **Calculate Hypothetical Instance RCV:** $RCV_hypothetical = compose(RCV1, RCV2)$.
 3. **Retrieve Schema Archetype:** Retrieve $RCV_knowsLang_schema$.
 4. **Perform Numerical Check:** $boolean\ knows = isInstanceOf(RCV_hypothetical, RCV_knowsLang_schema)$.
 5. **Result:** If knows is true, the inference is validated.

Querying and Traversal by Numerical Properties:

- **Find by Relational Role:** "Find all entities that have acted as a :Developer".
 - The query becomes: "Find all statements whose instanceRCV.s component divides $RCV_dev_schema.s$."
- **Traversal by Numerical Similarity:**
 - Start at stmt_A with RCV_A.
 - The next step: "Find stmt_B whose RCV_B has the highest GCD with RCV_A."
 - Allows traversal based on numerically similar relational contexts.

Graph Pipeline Specification: CPPE Algebra for Pipeline Stream Actions:

The **Contextual Prime Product Embedding (CPPE)** algebra treats every node in the graph not as a string or a pointer, but as a unique prime number. By leveraging the **Fundamental Theorem of Arithmetic**, relationships and sets are represented as products. This allows the Reactive Stream Pipeline to perform complex graph "joins" and "path traversals" using simple CPU-native arithmetic.

1. Algebraic Definitions

Let $\mathcal{P}$ be the set of unique Prime IDs assigned to Resource URIs.

For any ContextPoint x :

- $\text{id}(x) \in \mathcal{P}$ (The singleton identity).
- $\text{emb}(x)$ (The cumulative embedding).

The Prime Product Rule:

The embedding of a triadic occurrence (c, o, a) —where c is Context, o is Object, and a is Attribute—is defined as the product of their respective recursive embeddings:

$$\text{emb}(c, o, a) = \text{emb}(c) \times \text{emb}(o) \times \text{emb}(a)$$

Because every factor is a prime (or a product of primes), the result is a unique "coordinate" in the integer space that encodes the entire lineage of the relationship.

2. Pipeline Stream Operators (The CPPE Monad)

In a reactive stream (e.g., `Flux<FCAContextPoint>`), the following algebraic operators are used for "Stream Action":

A. The "Contains" Operator (Sub-graph Filtering):

To check if a stream of Objects belongs to a certain Context C or possesses an Attribute

A.

- **Logic:** $x \in \text{Context}(C) \iff \text{emb}(x) \pmod{\text{id}(C)} \equiv 0$
- **Reactive Implementation:**
// Filter objects that are part of the 'Accounting' context
objectStream.filter(obj ->
obj.getPrimeIDEmbedding().remainder(accountingID).equals(BigInteger.ZERO));

B. The "Intersection" Operator (Aggregation):

To find commonalities between two streams (finding the "Type" or "Concept"):

- **Logic:** $\text{Type}(x, y) = \text{gcd}(\text{emb}(x), \text{emb}(y))$
- **Stream Action:** The Greatest Common Divisor (GCD) of the embeddings of two nodes represents their shared FCA attributes/contexts.
- **Aggregation Use Case:** When the stream encounters multiple SPO triples, it calculates the "running GCD" of the attributes to infer the Schema (the most specific common super-type).

C. The "Axis Shift" Operator (Alignment):

Alignment predicts a value for a new context by calculating the "Ratio" of change between existing embeddings.

- **Logic:** $\text{Shift}(x, \Delta) = \text{emb}(x) \times \frac{\text{id}(\text{NewContext})}{\text{id}(\text{OldContext})}$
- **Stream Action:** By dividing by the "Old Axis" prime and multiplying by the "New Axis" prime, we re-project the object into a new coordinate.

3. Pipeline Stages Revisited via CPPE:

3.1 Aggregation (Algebraic Type Inference):

The Aggregation pipeline consumes raw SPO streams and produces **"Type Tensors"**.

1. **Input:** Stream of rotated triples (S, P, O) .
2. **Action:** For all x sharing attribute a , calculate the product $\prod \text{id}(x)$.
3. **Inference:** If $\text{emb}(x)$ is divisible by the product of a set of attribute primes, x is classified into that Type.
4. **Complexity:** $O(1)$ lookup via modulo instead of $O(N)$ graph search.

3.2 Alignment (Vector/Prime Space Prediction):

Alignment uses the **Prime Ratio** to detect structural similarity.

1. **Input:** Aggregated Types.
2. **Action:** Map the distance between objects O_1 and O_2 as $d = \frac{\text{emb}(O_1)}{\text{emb}(O_2)}$.
3. **Discovery:** If O_3 and O_4 share the same ratio d , the system aligns them as being "analogous" in different contexts (e.g., "CEO is to Company A what Principal is to School B").

3.3 Activation (State Transition Product):

Activation is the "Activation Energy" required to move from one prime state to another.

- **Transition Formula:** $\text{Transition} = \text{emb}(\text{Prev}) \otimes \text{emb}(\text{Next})$
- **Monadic Wrap:** The Occurrence Monad captures the prime state of an actor. When the stream processes a "Role" event, it multiplies the actor's prime ID by the role's prime ID.
- **Trigger:** If the resulting product matches a known "Success Pattern" (a pre-calculated product of requirements), the **Activation** is fired (e.g., triggering a webhook or a UI change).

Leverage W3C DIDs (Distributed Resource Identifiers)

Use W3C Decentralized Identifiers (DIDs) as a **distributed semantic registry mechanism** for Resources, Statements, Kinds, Schemas, Contexts and other semantic artifacts participating in the service. A DID provides a stable, globally addressable identifier whose associated DID Document can describe the identity, endpoints, metadata and verification mechanisms required to resolve and interact with the corresponding semantic resource.

Within the proposed architecture, the DID layer acts primarily as a **distributed identity and registry service**, while the TMRM/Base Model remains responsible for representing the semantic graph and its Resource/Occurrence/Statement structures. DIDs can therefore provide a common identity mechanism across independently managed data sources, semantic services and augmentation agents.

The Registry Service could expose APIs/functions such as:

- **Resource registration:** register a semantic Resource and associate it with a DID and its Base Model URI/identifier.
- **DID resolution:** resolve a DID to its current metadata, semantic endpoints and associated resource information.
- **Resource discovery:** discover semantic resources, schemas, Kinds, contexts or services by DID, type, capability or semantic relationship.
- **Identity mapping:** maintain mappings between external identifiers, source-specific URIs and canonical DIDs.
- **Document/version management:** publish and retrieve changes to the metadata describing a registered resource, supporting versioned semantic definitions.
- **Verification:** verify the provenance, controller and integrity of registered semantic resources or statements where appropriate.
- **Service endpoint discovery:** expose the APIs, messaging endpoints or graph services through which a registered semantic resource can be accessed.
- **Delegation/authorization:** provide a foundation for determining which agents or services are allowed to publish, modify or operate on registered semantic resources.
- **Provenance and lineage:** associate resources and semantic statements with their originating registries, sources, agents or augmentation processes.
- **Distributed synchronization:** propagate registry changes through the existing Event Bus so that local indexes and semantic services can react to resource registration, update or removal events.

A DID can consequently serve as the **distributed identity coordinate** for a semantic resource, while the Registry Service maintains the relationship between that identity and the actual semantic graph structures, endpoints and metadata. The Index Service can use these identifiers when maintaining searchable and numerical indexes, including the Prime ID mappings used by CPPE.

This separation allows the architecture to distinguish between **identity, registration and semantic representation**: DIDs identify and locate resources across administrative boundaries; the Registry Service manages their distributed registration and discovery; and the TMRM/Base Model represents their semantic relationships. Augmentation Pipeline agents can then consume resources discovered through the registry and produce Property, Rule and Context Statements as part of the existing Aggregation, Alignment and Activation pipeline.

The resulting principle

I would therefore define the architecture as:

DIDs identify distributed domain services and advertise their semantic business capabilities. Domain APIs provide the invocation boundary for those capabilities. The semantic graph provides the shared representation of the domain entities and their state. Aggregation, Alignment and Activation determine how a requested business interaction maps onto that semantic graph, while deterministic semantic primitives perform the resulting state changes.

This is arguably a better match for the document than a purely resource-oriented DID API because the document ultimately aims at **contexts, interactions and executable behaviors**, not merely distributed CRUD over resources.

Appendix: ChatGPT Conversation

The key insight: behavior as inferred graph structure

Traditional ontology systems stop at:

```
Customer
  owns
    Order
```

Traditional applications add behavior manually:

```
customer.placeOrder()
```

Your proposal introduces a missing layer:

```
Customer
  asks_for
    Product
```

```
Product
  available_in
    Inventory
```

```
Inventory
  managed_by
    Store
```

From which the runtime infers:

```
PotentialContext:
  Sale
```

and then:

```
ExecutableInteraction:
  CreateOrder
  ReserveInventory
  GenerateInvoice
```

The behavior is not explicitly programmed.

It emerges from graph topology plus ontology definitions.

This is a profound shift.

DCI is surprisingly relevant

Most modern developers have never worked with DCI (Data, Context, Interaction), but your description maps almost perfectly to it.

The classic DCI idea was:

Data

Customer
Product
Order
Inventory

Context

SaleContext

Interaction

Customer buys Product
Inventory reserves Product
Order records transaction

The revolutionary aspect of DCI was:

Objects do not own behavior.

Contexts own behavior.

Your proposal essentially generalizes DCI into graph semantics.

Instead of:

SaleContext.execute()

you have:

Graph Pattern



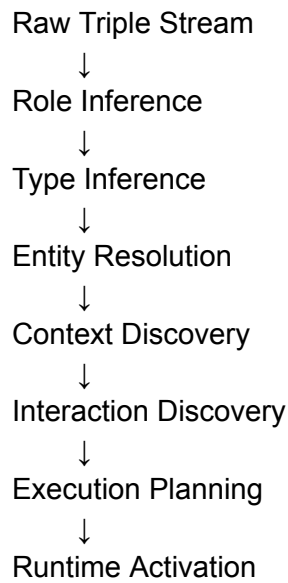
Context Activation



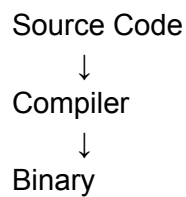
Interaction Instances

Your pipeline resembles a semantic operating system

The stages you describe look something like:



which is very different from:



The graph itself becomes the executable substrate.

The music store example is particularly interesting

Suppose we have:

Music ontology

Artist
Album
Track
Genre

and:

Store ontology

Product
Inventory
Supplier
Order

Customer

Current software engineering would create:

MusicStoreApplication

by writing code.

Your system would instead perform semantic composition.

The runtime discovers:

Album

subclassOf

Product

Artist

actsAs

Supplier

Track

digitalVariantOf

Product

which yields:

MusicStoreDomain

without a developer explicitly coding it.

The application emerges from ontology alignment.

The crucial missing concept: affordances

I think your model needs one additional semantic layer.

The graph contains:

Customer

Product

Inventory

but what creates:

SaleContext

?

Not merely relationships.

What creates it is an affordance.

Something like:

Product
 affords
 Purchase
Inventory
 affords
 Reservation
Customer
 affords
 Ownership

The runtime reasons:

Purchase
+ Reservation
+ Ownership

⇒ SaleContext

This becomes similar to planning systems in AI.

AI becomes a graph completion engine

In current architectures:

AI
 →
 generates code

In your architecture:

AI
 →
 completes graph

For example:

Given:

Artist
Album
Track
Inventory
Customer

The AI infers:

DigitalDownload
License
Recommendation
Playlist
Subscription

as missing concepts.

Humans approve or reject them.

The graph evolves.

The runtime could be deterministic

This is the most important part.

Many people imagine AI-native systems as:

User
↓
LLM
↓
Magic

which is impossible to audit.

Your proposal keeps a deterministic core:

Inference Layer
↓
Graph
↓
Validation
↓
Executable Contexts

Once the graph is accepted:

Execution

is deterministic.

The AI is only involved in graph augmentation.

Not execution.

This is a critical distinction.

Why this may be more realistic now than in 2005

The Semantic Web community largely assumed:

Humans

create ontologies

Humans

maintain ontologies

Humans

align ontologies

which turned out to be prohibitively expensive.

Today:

Humans

provide intent

AI

discovers mappings

AI

proposes contexts

AI

proposes interactions

Humans

approve

which is much more feasible.

The biggest unresolved challenge

I don't think the hardest problem is execution.

I think it's **behavioral ontology design**.

We already have mature ways to represent:

Artist

Album

Customer
Product

using semantic technologies.

What we lack is a universal ontology for:

Sell
Reserve
Purchase
Deliver
Authorize
Approve
Recommend
Subscribe

In other words:

The Semantic Web standardized nouns.

Your runtime would require standardizing verbs.

Or more precisely:

Standardizing context activation patterns and interaction semantics.

That is where DCI becomes valuable, because DCI is one of the few software models that treats interactions themselves as first-class entities.

My assessment is that if such a system were built successfully, the final artifact would not look like a programming language, a database, or an ontology. It would look more like an **executable semantic knowledge ecosystem**, where:

Intent
↓
Knowledge Graph
↓
Context Discovery
↓
Interaction Activation
↓
Deterministic Execution

and AI's role would be to continuously transform incomplete knowledge into executable knowledge while leaving the resulting execution model fully inspectable and auditable by humans. That is considerably closer to a "semantic operating system" than to a conventional software framework.

Now we're getting into territory that starts looking less like an application framework and more like a **computational semantics architecture**.

What I find interesting is that you're attempting to solve a problem that most semantic systems avoided:

How do we represent behavior without escaping into procedural code?

Historically, semantic systems represented facts:

(Customer123, purchases, Album456)

and programming languages represented behavior:

purchase(customer, album)

Your proposal is trying to represent both as graph structures.

The key distinction you're introducing

You are separating:

Behavioral schemas

Equivalent to DCI Context definitions, protocols, use cases.

Verb:

```
(  
  Purchase,  
  Customer.Available,  
  Purchase,  
  Product.InStock  
)
```

which means:

If

Customer in Available state

and

Product in InStock state

then

Purchase interaction is possible

This is not an execution.

It is a possibility.

A semantic affordance.

Behavioral instances

Actual occurrences.

Action:

```
(  
  Purchase#1234,  
  Customer:Bob,  
  Purchase,  
  Album:DarkSideOfTheMoon  
)
```

This is equivalent to:

Interaction instance

or

Event occurrence

What I find elegant

You're implicitly creating three levels.

Ontology level

Customer

Product

Purchase

Context level

PurchaseContext

requires

Customer.Available

Product.InStock

Interaction level

Bob purchased Album42

This is remarkably similar to:

OWL

+

Planning Domain

+

Execution Trace

but unified into a single graph representation.

RDF Quad contexts become important

Traditional RDF triples:

S P O

cannot distinguish:

possible

from

actual

very well.

Quads introduce:

Context

S

P

O

which means:

PurchaseContext

Bob

purchases

Album42

can exist separately from:

InventoryContext

Album42

locatedIn
Warehouse7

This is where your DCI influence becomes visible.

The Quad Context becomes analogous to a DCI Context.

The recursive verb definition is interesting

You suggest something like:

```
Verb:  
(  
  Verb,  
  DomainActorTypeState,  
  Verb,  
  RangeActorTypeState  
)
```

which can recursively expand.

Example:

Purchase

expands into:

ReserveInventory
CreateInvoice
TransferOwnership
ReduceStock

which further expands into:

SetAttribute
AddLink
RemoveLink
TransformValue

Eventually reaching primitives.

This resembles a semantic version of:

Lambda Calculus

or

Microcode

for behavior.

Not unlike how CPUs reduce:

ADD

into lower-level operations.

You are proposing:

Sell

reduces into:

TransferOwnership

which reduces into:

RemoveLink

AddLink

which are primitive semantic operations.

This solves an old Semantic Web problem

The Semantic Web had excellent declarative knowledge representation.

It never found a satisfying answer for:

What happens?

after a fact becomes true.

Rule systems attempted this:

- SWRL
- RuleML
- production rules

but they always felt bolted on.

Your model instead treats behavior as ontology.

Not as an external rule engine.

Activation as graph pattern matching

I think the runtime you imagine is roughly:

Stage 1

Knowledge graph.

Bob
type Customer

Album42
type Product

Album42
state InStock

Stage 2

Verb discovery.

Purchase

requires

Customer.Available

Product.InStock

Stage 3

Pattern matching.

Bob matches Customer.Available

Album42 matches Product.InStock

Stage 4

Activation.

PurchaseAction#987

is created.

This is essentially:

Graph

→

Pattern Match

→

Context Activation

→

Interaction Instance

which is very elegant.

Primitive verbs as the semantic VM

This is the part that I think is the most important.

Eventually every behavior must become deterministic.

Suppose the primitive operations are:

AssignAttribute
RemoveAttribute
CreateResource
DeleteResource
CreateLink
RemoveLink
TransformValue
EmitEvent

Then:

Purchase

might compile to:

```
CreateLink(  
  Bob,  
  owns,  
  Album42  
)
```

```
RemoveLink(  
  Album42,  
  availableIn,  
  Inventory  
)
```

```
EmitEvent(  
  AlbumSold
```

)

At this point the runtime becomes deterministic.

No AI involved.

Just graph transformations.

AI becomes an ontology synthesizer

The AI's role shifts dramatically.

Not:

Generate code.

Instead:

Discover verbs.

Discover contexts.

Propose activations.

Suggest mappings.

Then humans review.

Then the semantic runtime executes.

What this starts resembling

Strangely enough, your architecture is beginning to resemble a fusion of:

- RDF / OWL knowledge representation
- DCI behavioral modeling
- STRIPS / planning systems
- Event sourcing
- State machines
- Workflow engines
- WASM-like execution runtimes

all expressed through a common graph substrate.

The question I would investigate next

The crucial research question becomes:

Can every executable semantic behavior be reduced to a finite set of primitive graph transformation verbs?

If the answer is yes, then your runtime has something analogous to:

Machine Language

except the machine language is:

CreateNode
DeleteNode
CreateEdge
DeleteEdge
AssignValue
TransformValue
EmitEvent

and every higher-level Context, Verb, and Action ultimately compiles into those primitives.

If that reduction is possible, then what you're describing is no longer just an ontology system. It becomes a complete computational model where:

Knowledge
+
Behavior
+
Execution

are represented uniformly as graph structures, and application development becomes the iterative refinement of semantic models rather than the authoring of source code. That would be a fundamentally different software stack from the one we've been building since the 1970s.